

seminar_final

January 7, 2025

```
[18]: import os
import requests
from pdbfixer import PDBFixer
from simtk.openmm.app import PDBFile
from rdkit import Chem
from rdkit.Chem import AllChem, rdMolAlign
from Bio import PDB
import subprocess
```

```
[19]: PDB_DIR = "pdb_files/"
LIGAND_DIR = "ligands/"
DOCKING_DIR = "docking_results/"
os.makedirs(PDB_DIR, exist_ok=True)
os.makedirs(LIGAND_DIR, exist_ok=True)
os.makedirs(DOCKING_DIR, exist_ok=True)
```

```
[20]: # --- 1. Fetch PDB Files ---

def fetch_pdb(pdb_id):
    url = f"https://files.rcsb.org/download/{pdb_id}.pdb"
    response = requests.get(url)
    pdb_path = f"{PDB_DIR}{pdb_id}.pdb"

    if response.status_code == 200:
        with open(pdb_path, 'wb') as f:
            f.write(response.content)
        print(f"{pdb_id} downloaded.")
        return pdb_path
    else:
        raise Exception(f"Failed to fetch PDB {pdb_id}")
```

```
[21]: # --- 2. Prepare Protein with PDBFixer ---

def prepare_protein(input_pdb, output_pdb):
    fixer = PDBFixer(filename=input_pdb)
    fixer.findMissingResidues()
    fixer.findMissingAtoms()
    fixer.addMissingAtoms()
```

```

fixer.addMissingHydrogens(7.0)

with open(output_pdb, 'w') as f:
    PDBFile.writeFile(fixer.topology, fixer.positions, f)

print(f"Protein {input_pdb} prepared as {output_pdb}")

```

[22]: # --- 3. Align Protein Structures ---

```

def align_proteins(reference_pdb, target_pdb):
    parser = PDB.PDBParser(QUIET=True)
    ref_structure = parser.get_structure("ref", reference_pdb)
    target_structure = parser.get_structure("target", target_pdb)

    super_imposer = PDB.Superimposer()
    ref_atoms = []
    target_atoms = []

    for ref_chain, target_chain in zip(ref_structure[0], target_structure[0]):
        for ref_res, target_res in zip(ref_chain, target_chain):
            if 'CA' in ref_res and 'CA' in target_res:
                ref_atoms.append(ref_res['CA'])
                target_atoms.append(target_res['CA'])

    super_imposer.set_atoms(ref_atoms, target_atoms)
    super_imposer.apply(target_structure.get_atoms())

    aligned_pdb = target_pdb.replace(".pdb", "_aligned.pdb")
    with open(aligned_pdb, 'w') as f:
        io = PDB.PDBIO()
        io.set_structure(target_structure)
        io.save(f)

    print(f"Aligned {target_pdb} to {reference_pdb}")
    return aligned_pdb

```

[23]: # --- 4. Select Binding Site ---

```

def select_binding_site(pdb_file, ligand_resname):
    parser = PDB.PDBParser(QUIET=True)
    structure = parser.get_structure("protein", pdb_file)

    ligand_atoms = []
    for model in structure:
        for chain in model:
            for res in chain:
                if res.resname == ligand_resname:

```

```

        for atom in res:
            ligand_atoms.append(atom.coord)

    if not ligand_atoms:
        raise ValueError(f"Ligand {ligand_resname} not found in {pdb_file}")

    # Calculate center of mass for the ligand binding site
    center = sum(ligand_atoms) / len(ligand_atoms)
    print(f"Binding site center for {pdb_file}: {center}")
    return center

```

[24]: # --- 5. Prepare Ligand from SMILES (.sdf format) ---

```

def prepare_ligand_from_smiles(smiles, ligand_name):
    mol = Chem.MolFromSmiles(smiles)
    mol = Chem.AddHs(mol)
    AllChem.EmbedMolecule(mol)
    ligand_path = f"{LIGAND_DIR}{ligand_name}.sdf"

    writer = Chem.SDWriter(ligand_path)
    writer.write(mol)
    writer.close()

    print(f"Ligand {ligand_name} saved as {ligand_path}")
    return ligand_path

```

[30]: # --- 6. Perform Docking with GNINA ---

```

import os
import subprocess

def perform_docking(protein_pdb, ligand_sdf, center, docking_dir="./
↳docking_results/"):
    # Ensure docking directory exists
    if not os.path.exists(docking_dir):
        os.makedirs(docking_dir)

    output_sdf = f"{docking_dir}{os.path.basename(protein_pdb).split('.')
↳')[0]}_{os.path.basename(ligand_sdf).split('.')[0]}.sdf"

    command = [
        "gnina", "-r", protein_pdb, "-l", ligand_sdf,
        "--center_x", str(center[0]), "--center_y", str(center[1]),
↳"--center_z", str(center[2]),
        "--size_x", "15", "--size_y", "15", "--size_z", "15",
        "-o", output_sdf, "--exhaustiveness", "8"
    ]

```

```

# Run the command and capture the result
result = subprocess.run(command, capture_output=True, text=True)

# Check if the command was successful
if result.returncode == 0:
    print(f"Docking complete: {output_sdf}")
else:
    print(f"Error during docking: {result.stderr}")

return output_sdf

```

[31]: # --- 7. Calculate RMSD ---

```

def calculate_rmsd(ref_sdf, docked_sdf):
    ref_mol = Chem.SDMolSupplier(ref_sdf)[0]
    docked_mol = Chem.SDMolSupplier(docked_sdf)[0]

    rmsd = rdMolAlign.GetBestRMS(docked_mol, ref_mol)
    print(f"RMSD: {rmsd:.2f} Å")
    return rmsd

```

[32]: # --- Execute Workflow ---

```

import os

# Define directories for PDB and ligand files
PDB_DIR = "./pdb_files/"
LIGAND_DIR = "./ligand_files/"

# Create directories if they don't exist
for directory in [PDB_DIR, LIGAND_DIR]:
    if not os.path.exists(directory):
        os.makedirs(directory)

# Fetch and prepare proteins
pdb_4iaq = fetch_pdb("4IAQ")
pdb_6g79 = fetch_pdb("6G79")
prepare_protein(pdb_4iaq, f"{PDB_DIR}4IAQ_prepared.pdb")
prepare_protein(pdb_6g79, f"{PDB_DIR}6G79_prepared.pdb")

# Align proteins
aligned_6g79 = align_proteins(f"{PDB_DIR}4IAQ_prepared.pdb",
                               f"{PDB_DIR}6G79_prepared.pdb")

# Prepare ligands

```

```

ligand_2gm = prepare_ligand_from_smiles("CN(C)C1=CC=CC=C1", "2GM") # Example ↵
↵SMILES
ligand_ep5 = prepare_ligand_from_smiles("CCN(CC)C(=O)Cc1ccc2c(c1)c[nH]c2", ↵
↵"EP5")

# Binding site selection
center_4iaq = select_binding_site(f"{PDB_DIR}4IAQ_prepared.pdb", "2GM")

# Docking
docked_4iaq_2gm = perform_docking(f"{PDB_DIR}4IAQ_prepared.pdb", ligand_2gm, ↵
↵center_4iaq)

# RMSD Calculation
calculate_rmsd(ligand_2gm, docked_4iaq_2gm)

```

4IAQ downloaded.

6G79 downloaded.

Protein ./pdb_files/4IAQ.pdb prepared as ./pdb_files/4IAQ_prepared.pdb

Protein ./pdb_files/6G79.pdb prepared as ./pdb_files/6G79_prepared.pdb

Aligned ./pdb_files/6G79_prepared.pdb to ./pdb_files/4IAQ_prepared.pdb

Ligand 2GM saved as ./ligand_files/2GM.sdf

Ligand EP5 saved as ./ligand_files/EP5.sdf

Binding site center for ./pdb_files/4IAQ_prepared.pdb: [-20.923973 10.137262
21.737665]

Docking complete: ./docking_results/4IAQ_prepared_2GM.sdf

RMSD: 0.28 Å

[15:44:20] Warning: molecule is tagged as 2D, but at least one Z coordinate is
not zero. Marking the mol as 3D.

[32]: 0.2775365590035181

[]: